

在 gRPC Java 中設定基本 OpenTelemetry 外掛程式

來源：<https://codelabs.developers.google.com/grpc/basic-otel-plugin-grpc-java?hl=zh-tw>

步驟數：6

如何為 gRPC Java 應用程式設定 OpenTelemetry 外掛程式

目錄

1. [簡介](#)
2. [事前準備](#)
3. [註冊 OpenTelemetry 外掛程式](#)
4. [執行範例並查看指標](#)
5. [在 Prometheus 中查看指標](#)
6. [\(選用\) 使用者練習](#)

步驟 1 · 約 5 分鐘

1. 簡介

在本程式碼研究室中，您將使用 gRPC 建立用戶端和伺服器，做為以 Java 編寫的路線對應應用程式基礎。

在本教學課程結束時，您將擁有一個簡單的 gRPC HelloWorld 應用程式，並使用 gRPC OpenTelemetry 外掛程式進行檢測，且能夠在 Prometheus 中查看匯出的可觀測性指標。

課程內容

- 如何為現有的 gRPC Java 應用程式設定 OpenTelemetry 外掛程式
 - 執行本機 Prometheus 執行個體
 - 將指標匯出至 Prometheus
 - 從 Prometheus 資訊主頁查看指標
-

步驟 2 · 約 15 分鐘

2. 事前準備

軟硬體需求

- `git`
- `curl`
- JDK v8 以上版本

設定前置條件：

```
sudo apt-get update -y
sudo apt-get upgrade -y
sudo apt-get install -y git curl
```

取得程式碼

為簡化學習過程，本程式碼研究室提供預先建構的原始碼架構，協助您踏出第一步。下列步驟會引導您在應用程式中檢測 gRPC OpenTelemetry 外掛程式。

[grpc-codelabs](#)

本程式碼研究室的架構原始碼位於這個 GitHub [目錄](#)。如果您不想自行導入程式碼，可以前往「completed」目錄查看已完成的原始碼。

首先，請複製 grpc Codelab 存放區，然後 cd 到 grpc-java-opentelemetry 資料夾：

□□□

```
git clone https://github.com/grpc-ecosystem/grpc-codelabs.git
cd grpc-codelabs/codelabs/grpc-java-opentelemetry/
```

或者，您也可以下載只包含 Codelab 目錄的 .zip 檔案，然後手動解壓縮。

步驟 3 · 約 7 分鐘

3. 註冊 OpenTelemetry 外掛程式

我們需要 gRPC 應用程式來新增 gRPC OpenTelemetry 外掛程式。在本程式碼研究室中，我們將使用簡單的 gRPC HelloWorld 用戶端和伺服器，並透過 gRPC OpenTelemetry 外掛程式進行檢測。

首先，請在用戶端中註冊已設定 Prometheus 匯出器的 OpenTelemetry 外掛程式。使用慣用的編輯器開啟

```
code-labs/grpc-java-opentelemetry/start_here/src/main/java/io/grpc/code-labs/opentelemetry/OpenTelemetryClient.java
```

，然後修改 main，新增程式碼來設定 gRPC Java OpenTelemetry API。

在用戶端設定插樁

建立 Prometheus 匯出工具

建立 PrometheusHttpServer，將 OpenTelemetry 指標轉換為 Prometheus 格式，並透過 HttpServer 公開這些指標。下列程式碼片段會建立新的 [Prometheus Exporter](#)。

```
// Default prometheus port i.e `prometheusPort` has been initialized to 9465

PrometheusHttpServer prometheusExporter = PrometheusHttpServer.builder()
    .setPort(prometheusPort)
    .build();
```

建立 OpenTelemetry SDK 執行個體

將上述建立的 `prometheusExporter` 註冊為 MetricReader，從 [SdkMeterProvider](#) 讀取指標。SdkMeterProvider 用於設定指標。

```
SdkMeterProvider sdkMeterProvider = SdkMeterProvider.builder()
    .registerMetricReader(prometheusExporter)
    .build();
```

使用上述建立的 `sdkMeterProvider` 建立 OpenTelemetrySdk 執行個體，以實作 [OpenTelemetry](#) 的 SDK。

```
OpenTelemetrySdk openTelemetrySdk = OpenTelemetrySdk.builder()
    .setMeterProvider(sdkMeterProvider)
    .build();
```

注意：系統會在 OpenTelemetry Meter Provider 上設定 Prometheus Exporter。(您也可以透過其他方式匯出指標。本程式碼研究室選擇使用 prometheus 匯出工具。) 這個 MeterProvider 會提供給 gRPC 的 OpenTelemetry 外掛程式做為設定。OpenTelemetry 外掛程式完成全域註冊後，所有 gRPC 用戶端和伺服器都會檢測 OpenTelemetry。

建立 GrpcOpenTelemetry 執行個體

使用 GrpcOpenTelemetry [API](#) 設定 OpenTelemetry SDK，該 SDK 會使用 Prometheus 指標匯出工具。

```
GrpcOpenTelemetry grpcOpenTelmetry = GrpcOpenTelemetry.newBuilder()  
    .sdk(openTelemetrySdk)  
    .build();  
  
// Registers gRPC OpenTelemetry globally.  
grpcOpenTelmetry.registerGlobal();
```

使用 `registerGlobal` 全域註冊 `GrpcOpenTelemetry` 執行個體後，後續建立的所有 gRPC 用戶端和伺服器都會使用 `OpenTelemetry` 進行檢測。

關閉 OpenTelemetry SDK

關機作業必須在 `ShutDownHook` 內進行。`openTelemetrySdk.close()` 會關閉 SDK，並在 `SdkMeterProvider` 上呼叫關機。

在伺服器上設定檢測

同樣地，我們也將 `GrpcOpenTelemetry` 新增至伺服器。開啟 `code-labs/grpc-java-opentelemetry/start_here/src/main/java/io/grpc/code-labs/opentelemetry/OpenTelemetryServer.java` 並新增程式碼，初始化 `GrpcOpenTelemetry`。

建立 Prometheus 匯出工具

由於這個程式碼研究室可能在同一部機器上執行，因此我們使用不同的通訊埠來代管 gRPC 伺服器端指標，避免在建立 `PrometheusHttpServer` 時發生通訊埠衝突。

```
// Default prometheus port i.e `prometheusPort` has been set to 9464  
  
PrometheusHttpServer prometheusExporter = PrometheusHttpServer.builder()  
    .setPort(prometheusPort)  
    .build();
```

建立 OpenTelemetry SDK 執行個體

```
SdkMeterProvider sdkMeterProvider = SdkMeterProvider.builder()  
    .registerMetricReader(prometheusExporter)  
    .build();
```

使用 OpenTelemetry SDK 初始化 GrpcOpenTelemetry

```
OpenTelemetrySdk openTelemetrySdk = OpenTelemetrySdk.builder()  
    .setMeterProvider(sdkMeterProvider)  
    .build();
```

建立 GrpcOpenTelemetry 執行個體

```
GrpcOpenTelemetry grpcOpenTelmetry = GrpcOpenTelemetry.newBuilder()  
    .sdk(openTelemetrySdk)  
    .build();  
    // Registers gRPC OpenTelemetry globally.  
grpcOpenTelmetry.registerGlobal();
```

關閉 OpenTelemetry SDK

gRPC 管道關閉後，呼叫 `openTelemetrySdk.close()` 會關閉 SDK，並在 `SdkMeterProvider` 上呼叫關機。

步驟 4 · 約 0 分鐘

4. 執行範例並查看指標

如要執行伺服器，請執行下列指令：

```
cd start_here
../gradlew installDist
./build/install/start_here/bin/opentelemetry-server
```

設定成功後，伺服器會顯示下列輸出內容：

```
[date and time] io.grpc.codelabs.opentelemetry.OpenTelemetryServer start
INFO: Server started, listening on 50051
```

伺服器執行期間，在另一個終端機中執行用戶端：

```
./build/install/start_here/bin/opentelemetry-client world
```

成功執行的結果如下所示：

```
[date and time]io.grpc.codelabs.opentelemetry.OpenTelemetryClient greet
INFO: Greeting: Hello world
[date and time] io.grpc.codelabs.opentelemetry.OpenTelemetryClient greet
INFO: Will try to greet world ...
[date and time]io.grpc.codelabs.opentelemetry.OpenTelemetryClient greet
INFO: Greeting: Hello world
```

我們已設定 gRPC OpenTelemetry 外掛程式，使用 Prometheus 匯出指標。這些指標會顯示在伺服器的 localhost:9464 和用戶端的 localhost:9465。

如要查看用戶端指標，請按照下列步驟操作：

```
curl localhost:9465/metrics
```

結果會採用以下形式：


```

# HELP grpc_client_attempt_duration_seconds Time taken to complete a client call attempt
# TYPE grpc_client_attempt_duration_seconds histogram
grpc_client_attempt_duration_seconds_bucket{grpc_method="helloworld.Greeter/SayHello",grpc_status="OK",grpc_target="dns:///localhost:50051",otel_scope_name="grpc-java",otel_scope_version="1.66.0",le="0.002"} 0
grpc_client_attempt_duration_seconds_bucket{grpc_method="helloworld.Greeter/SayHello",grpc_status="OK",grpc_target="dns:///localhost:50051",otel_scope_name="grpc-java",otel_scope_version="1.66.0",le="0.003"} 2
grpc_client_attempt_duration_seconds_bucket{grpc_method="helloworld.Greeter/SayHello",grpc_status="OK",grpc_target="dns:///localhost:50051",otel_scope_name="grpc-java",otel_scope_version="1.66.0",le="0.004"} 14
grpc_client_attempt_duration_seconds_bucket{grpc_method="helloworld.Greeter/SayHello",grpc_status="OK",grpc_target="dns:///localhost:50051",otel_scope_name="grpc-java",otel_scope_version="1.66.0",le="0.005"} 29
grpc_client_attempt_duration_seconds_bucket{grpc_method="helloworld.Greeter/SayHello",grpc_status="OK",grpc_target="dns:///localhost:50051",otel_scope_name="grpc-java",otel_scope_version="1.66.0",le="0.1"} 33
grpc_client_attempt_duration_seconds_bucket{grpc_method="helloworld.Greeter/SayHello",grpc_status="OK",grpc_target="dns:///localhost:50051",otel_scope_name="grpc-java",otel_scope_version="1.66.0",le="+Inf"} 34
grpc_client_attempt_duration_seconds_count{grpc_method="helloworld.Greeter/SayHello",grpc_status="OK",grpc_target="dns:///localhost:50051",otel_scope_name="grpc-java",otel_scope_version="1.66.0"} 34
grpc_client_attempt_duration_seconds_sum{grpc_method="helloworld.Greeter/SayHello",grpc_status="OK",grpc_target="dns:///localhost:50051",otel_scope_name="grpc-java",otel_scope_version="1.66.0"} 0.46512665300000006
# HELP grpc_client_attempt_rcvd_total_compressed_message_size_bytes Compressed message bytes received per call attempt
# TYPE grpc_client_attempt_rcvd_total_compressed_message_size_bytes histogram
grpc_client_attempt_rcvd_total_compressed_message_size_bytes_bucket{grpc_method="helloworld.Greeter/SayHello",grpc_status="OK",grpc_target="dns:///localhost:50051",otel_scope_name="grpc-java",otel_scope_version="1.66.0",le="0.0"} 0
grpc_client_attempt_rcvd_total_compressed_message_size_bytes_sum{grpc_method="helloworld.Greeter/SayHello",grpc_status="OK",grpc_target="dns:///localhost:50051",otel_scope_name="grpc-java",otel_scope_version="1.66.0"} 442.0
# HELP grpc_client_attempt_sent_total_compressed_message_size_bytes Compressed message bytes sent per client call attempt
# TYPE grpc_client_attempt_sent_total_compressed_message_size_bytes histogram
grpc_client_attempt_sent_total_compressed_message_size_bytes_bucket{grpc_method="helloworld.Greeter/SayHello",grpc_status="OK",grpc_target="dns:///localhost:50051",otel_scope_name="grpc-java",otel_scope_version="1.66.0",le="0.0"} 0
grpc_client_attempt_sent_total_compressed_message_size_bytes_bucket{grpc_method="helloworld.Greeter/SayHello",grpc_status="OK",grpc_target="dns:///localhost:50051",otel_scope_name="grpc-java",otel_scope_version="1.66.0",le="1024.0"} 34
grpc_client_attempt_sent_total_compressed_message_size_bytes_sum{grpc_method="helloworld.Greeter/SayHello",grpc_status="OK",grpc_target="dns:///localhost:50051",otel_scope_name="grpc-java",otel_scope_version="1.66.0"} 238.0
# HELP grpc_client_attempt_started_total Number of client call attempts started
# TYPE grpc_client_attempt_started_total counter
grpc_client_attempt_started_total{grpc_method="helloworld.Greeter/SayHello",grpc_target="dns:///localhost:50051",otel_scope_name="grpc-java",otel_scope_version="1.66.0"} 34.0
# HELP grpc_client_call_duration_seconds Time taken by gRPC to complete an RPC from

```

```
application's perspective
# TYPE grpc_client_call_duration_seconds histogram
grpc_client_call_duration_seconds_bucket{grpc_method="helloworld.Greeter/SayHello",grpc_status="OK",grpc_target="dns:///localhost:50051",otel_scope_name="grpc-java",otel_scope_version="1.66.0",le="0.0"} 0
grpc_client_call_duration_seconds_bucket{grpc_method="helloworld.Greeter/SayHello",grpc_status="OK",grpc_target="dns:///localhost:50051",otel_scope_name="grpc-java",otel_scope_version="1.66.0",le="0.003"} 2
grpc_client_call_duration_seconds_bucket{grpc_method="helloworld.Greeter/SayHello",grpc_status="OK",grpc_target="dns:///localhost:50051",otel_scope_name="grpc-java",otel_scope_version="1.66.0",le="+Inf"} 34
grpc_client_call_duration_seconds_count{grpc_method="helloworld.Greeter/SayHello",grpc_status="OK",grpc_target="dns:///localhost:50051",otel_scope_name="grpc-java",otel_scope_version="1.66.0"} 34
grpc_client_call_duration_seconds_sum{grpc_method="helloworld.Greeter/SayHello",grpc_status="OK",grpc_target="dns:///localhost:50051",otel_scope_name="grpc-java",otel_scope_version="1.66.0"} 0.512708707
# TYPE target_info gauge
target_info{service_name="unknown_service:java",telemetry_sdk_language="java",telemetry_sdk_name="opentelemetry",telemetry_sdk_version="1.40.0"} 1
```

同樣地，伺服器端指標也是如此：

```
curl localhost:9464/metrics
```

步驟 5 · 約 5 分鐘

5. 在 Prometheus 中查看指標

在這裡，我們將設定 Prometheus 執行個體，用於擷取 gRPC 範例用戶端和伺服器，這些用戶端和伺服器會使用 Prometheus 匯出指標。

下載適用於您平台的最新版 [Prometheus](#)，然後解壓縮並執行：

```
tar xvfz prometheus-*.tar.gz
cd prometheus-*
```

建立 Prometheus 設定檔，並加入下列內容：

```
cat > grpc_otel_java_prometheus.yml <<EOF
scrape_configs:
  - job_name: "prometheus"
    scrape_interval: 5s
    static_configs:
      - targets: ["localhost:9090"]
  - job_name: "grpc-otel-java"
    scrape_interval: 5s
    static_configs:
      - targets: ["localhost:9464", "localhost:9465"]
EOF
```

使用新設定啟動 Prometheus：

```
./prometheus --config.file=grpc_otel_java_prometheus.yml
```

這會設定每 5 秒擷取一次用戶端和伺服器程式碼實驗室程序的指標。

前往 <http://localhost:9090/graph> 查看指標。例如，查詢 -

```
histogram_quantile(0.5, rate(grpc_client_attempt_duration_seconds_bucket[1m]))
```

會顯示圖表，其中以 1 分鐘做為分位數計算的時間視窗，顯示嘗試延遲時間中位數。

查詢率 -

```
increase(grpc_client_attempt_duration_seconds_bucket[1m])
```

步驟 6 · 約 5 分鐘

6. (選用) 使用者練習

在 Prometheus 資訊主頁中，您會發現 QPS 偏低。看看您是否能在範例中找出限制 QPS 的可疑程式碼。

對於熱衷於此的開發人員，用戶端程式碼會限制在特定時間點只能有一個待處理的 RPC。您可以修改這項設定，讓用戶端傳送更多 RPC，不必等待先前的 RPC 完成。(這項問題的解決方案尚未提供)。
